



Python Programming Fundamentals

pytest Fixtures and Advanced Features

Programming Fundamentals Team

SETU

2026-06-12



Outline

1. The Problem with Repeated Setup
2. `@pytest.fixture`
3. Fixture Scope
4. `conftest.py`
5. `@pytest.mark.parametrize`
6. Testing Exceptions with `pytest.raises`
7. Code Coverage Overview



Outline

1. The Problem with Repeated Setup

2. `@pytest.fixture`

3. Fixture Scope

4. `conftest.py`

5. `@pytest.mark.parametrize`

6. Testing Exceptions with `pytest.raises`

7. Code Coverage Overview



1. The Problem with Repeated Setup

Without fixtures – repetitive!

```
def test_deposit():
    account = BankAccount("Alice", 100.0)    # repeated
    account.deposit(50.0)
    assert account.get_balance() == 150.0

def test_withdraw():
    account = BankAccount("Alice", 100.0)    # repeated
    account.withdraw(30.0)
    assert account.get_balance() == 70.0

def test_balance():
    account = BankAccount("Alice", 100.0)    # repeated again
    assert account.get_balance() == 100.0
```



Outline

1. The Problem with Repeated Setup
2. **`@pytest.fixture`**
3. Fixture Scope
4. `conftest.py`
5. `@pytest.mark.parametrize`
6. Testing Exceptions with `pytest.raises`
7. Code Coverage Overview



2. @pytest.fixture

```
import pytest
from bank_account import BankAccount

@pytest.fixture
def account():
    """Provides a fresh BankAccount for each test."""
    return BankAccount("Alice", 100.0)

# Use it as a parameter – pytest injects it automatically
def test_deposit(account):
    account.deposit(50.0)
    assert account.get_balance() == 150.0

def test_withdraw(account):
    account.withdraw(30.0)
    assert account.get_balance() == 70.0
```



2. `@pytest.fixture`

Each test gets a **fresh** account — changes in one test don't affect others.



Outline

1. The Problem with Repeated Setup
2. `@pytest.fixture`
- 3. Fixture Scope**
4. `conftest.py`
5. `@pytest.mark.parametrize`
6. Testing Exceptions with `pytest.raises`
7. Code Coverage Overview



3. Fixture Scope

```
@pytest.fixture(scope="function") # default – new per test
```

```
def account():  
    return BankAccount("Alice", 100.0)
```

```
@pytest.fixture(scope="module") # shared across all tests in file
```

```
def db_connection():  
    conn = setup_db()  
    yield conn # "yield" fixtures support teardown  
    conn.close() # teardown – runs after all tests in module
```

yield fixtures are useful for setup/teardown:

```
@pytest.fixture
```

```
def temp_shop(tmp_path):  
    shop = Shop("Test")  
    shop.add_product(Product("Apple", 1.99, 101))
```



3. Fixture Scope

```
yield shop  
# teardown: nothing needed – tmp_path is cleaned automatically
```



Outline

1. The Problem with Repeated Setup
2. `@pytest.fixture`
3. Fixture Scope
4. **`conftest.py`**
5. `@pytest.mark.parametrize`
6. Testing Exceptions with `pytest.raises`
7. Code Coverage Overview



4.1 Shared fixtures

Create tests/conftest.py — fixtures here are available to **all test files** in the folder:

```
# tests/conftest.py
import pytest
from bank_account import BankAccount
from product import Product
from shop import Shop

@pytest.fixture
def account():
    return BankAccount("Alice", 100.0)

@pytest.fixture
def product():
    return Product("Apple", 1.99, 101)
```



4.1 Shared fixtures

```
@pytest.fixture
def populated_shop():
    shop = Shop("Test Shop")
    shop.add_product(Product("Apple", 1.99, 101))
    shop.add_product(Product("Bread", 2.50, 102))
    shop.add_product(Product("Milk", 1.20, 103))
    return shop
```



Outline

1. The Problem with Repeated Setup

2. `@pytest.fixture`

3. Fixture Scope

4. `conftest.py`

5. `@pytest.mark.parametrize`

6. Testing Exceptions with `pytest.raises`

7. Code Coverage Overview



5.1 Test multiple inputs with one function

```
import pytest
from product import Product

@pytest.mark.parametrize("name, price, pid", [
    ("Apple", 1.99, 101),
    ("Bread", 2.50, 102),
    ("Milk", 1.20, 103),
])
def test_product_creation(name, price, pid):
    p = Product(name, price, pid)
    assert p.get_name() == name
    assert p.get_price() == price
    assert p.get_product_id() == pid
```

This runs 3 tests automatically — one per row.



Outline

1. The Problem with Repeated Setup
2. @pytest.fixture
3. Fixture Scope
4. conftest.py
5. @pytest.mark.parametrize
- 6. Testing Exceptions with pytest.raises**
7. Code Coverage Overview



6. Testing Exceptions with `pytest.raises`

```
import pytest
from product import Product

def test_negative_price_raises():
    with pytest.raises(ValueError):
        Product("Apple", -1.0, 101)

def test_zero_price_raises():
    with pytest.raises(ValueError):
        Product("Apple", 0.0, 101)

# Check the error message
def test_negative_price_message():
    with pytest.raises(ValueError, match="greater than 0"):
        Product("Apple", -1.0, 101)
```



Outline

1. The Problem with Repeated Setup
2. `@pytest.fixture`
3. Fixture Scope
4. `conftest.py`
5. `@pytest.mark.parametrize`
6. Testing Exceptions with `pytest.raises`
- 7. Code Coverage Overview**



7. Code Coverage Overview

```
# Install
```

```
pip install pytest-cov
```

```
# Run with coverage
```

```
pytest --cov=. --cov-report=term-missing
```

Output:

Name	Stmts	Miss	Cover	Missing

product.py	28	2	93%	45, 52
bank_account.py	18	0	100%	

TOTAL	46	2	96%	

- Aim for > 80% coverage
- Lines 45 and 52 are not tested — write tests for them!



Thanks for Watching - Any questions?

